Tu equipo

Equipo 05 @user-bcp-student47 Integrante

Actualizar equipo

Tu rol

backend · user-bcp-student09

Equipo asignado por el administrador.

Completado

S1 · Paso 1 de 22

Revisa tu equipo y prepara el entorno

Revisa tu equipo, tu líder asignado y tu participación. Comprueba tu entorno en Bash. Los equipos tienen seis perfiles; solo el equipo de siete añade Datos y persistencia en CrewAI, con su propio archivo agents/data.py.

Todo el equipo

Cómo realizar este paso

Comprobar herramientas en Bash

Confirma que tienes Python 3.12, Git y GitHub CLI.

Comando

```
python --version
git --version
gh --version
```

Sesión de GitHub CLI

GitHub CLI crea, clona y abre los PR del equipo. Debe estar autenticado con tu cuenta personal.

Comando

```
gh auth status
```

Completado

S1 · Paso 2 de 22

Líder: crear crewai desde la plantilla

El líder crea una sola vez el repo crewai en su cuenta personal a partir de NTTData-Academy/crewai. Si ya existe, verifica que sea el repo del equipo y reutilízalo. Los integrantes esperan al líder y crean un fork de su repo.

Solo el líder

Cómo realizar este paso

Este paso lo hace tu líder @user-bcp-student47. Revisa las instrucciones de este paso y espera a que lo complete para continuar con tu equipo.

Completado

S1 · Paso 3 de 22

Elegir tu rol y crear tu fork

Cada persona guarda su rol en «Antes de empezar». El líder ya creó crewai desde la plantilla en el paso anterior y usa ese repositorio. Cada integrante crea un fork del repositorio del líder en su propia cuenta de GitHub.

Líder e integrantes

Cómo realizar este paso

Cómo trabajamos en equipo

El líder también toma un rol. Cada persona entrega un PR que modifica solo su archivo. Solo el equipo de siete añade agents/data.py en CrewAI y Guardrails; su responsable diseña datos y persistencia. El líder configura CREWAI_INCLUDE_DATA=true en su .env antes de una ejecución nueva. Datos se ejecuta después de arquitectura y antes de backend, con aprobación humana. El laboratorio guarda los responsables al elegir el rol. En Guardrails el líder configura GUARDRAILS_INCLUDE_DATA=true; los equipos de seis mantienen el flujo habitual.

Nota

architect → agents/architect.py: Define componentes, límites y contrato de API: entradas, respuestas y errores, alineados con los criterios de aceptación.
data → agents/data.py: Define tablas, relaciones, restricciones y migraciones: claves primarias y foráneas, nulabilidad, unicidad, índices e integridad; conserva los datos existentes y propone reversión o recuperación. Solo para el equipo de siete en CrewAI y Guardrails.
backend → agents/backend.py: Implementa servicios, endpoints, validaciones y acceso a datos según los contratos aprobados de Arquitectura y Datos. En equipos de seis también cubre la persistencia.
frontend → agents/frontend.py: Implementa la interacción y el consumo de API, con estados de carga, vacío, error y accesibilidad.
testing → agents/testing.py: Convierte criterios de aceptación en pruebas positivas, negativas, de borde e integración; propone comandos y distingue resultados reales de NO EJECUTADO.
security → agents/security.py: Revisa autenticación, autorización, entradas, secretos, datos sensibles e inyección; entrega hallazgos, correcciones y comprobaciones.
chapter → agents/chapter.py: Contrasta requerimiento, arquitectura, datos, código, pruebas y seguridad; emite un veredicto con motivos y correcciones. Conserva las aprobaciones humanas.

Completado

S1 · Paso 4 de 22

Clonar crewai, preparar Python y ubicar el producto

El líder usa su repo crewai y cada integrante su fork del líder. Abre una nueva terminal desde el acceso Workshop - Bash creado el Día 1. Empiezas en tu carpeta del curso, <matricula>-<nombre>/. Ahí clonarás crewai, junto a reqflow-workspace/.

Cada persona

Cómo realizar este paso

Dónde queda cada repositorio

Esta es la estructura que tendrás al clonar los dos repositorios del Día 4. Ahora vas a trabajar con crewai/. Cada clon tiene su propio .venv y .env. Conserva los productos dentro de reqflow-workspace/.

Nota

```
<matricula>-<nombre>/
├── reqflow-workspace/
│   └── repositorios del producto de los días anteriores
├── crewai/
└── guardrails/
```

Clonar el repo de mi cuenta

Ejecuta este bloque desde tu carpeta <matricula>-<nombre>/. Descarga crewai desde tu cuenta de GitHub, entra en el clon y muestra su ruta. Si la descarga falla, el bloque se detiene.

Comando

```
gh repo clone user-bcp-student09/crewai &&
cd crewai &&
```

```
pwd
```

Conectar mi fork con el repo del líder

origin es tu fork para hacer push. upstream es el repo del líder, destino de los PR. Si upstream ya existe, comprueba su URL con git remote -v en lugar de añadirlo otra vez.

Comando

```
git remote get-url upstream >/dev/null 2>&1 || git remote add upstream
https://github.com/user-bcp-student47/crewai.git
git remote -v
```

Crear el entorno en Bash

Crea .venv con Python 3.12 solo si falta, activa el entorno existente, instala las dependencias y copia .env.example solo si aún no tienes .env.

Comando

```
[ -d .venv ] || python -m venv .venv
if [ -f .venv/Scripts/activate ]; then
  source .venv/Scripts/activate
else
  source .venv/bin/activate
fi
python -m pip install -r requirements.txt
[ -f .env ] || cp .env.example .env
```

Ubicar el producto de los días anteriores

Desde la raíz del clon de esta sesión, comprueba el producto existente. workshop-auth-backend es el ejemplo: sustituye el nombre por el repo que corresponde al

issue. Conserva su historial y los cambios de los días previos; este paso no crea ni clona otro producto.

Comando

```
git -C ../reqflow-workspace/workshop-auth-backend remote -v
git -C ../reqflow-workspace/workshop-auth-backend status --short --branch
```

Completado

S1 · Paso 5 de 22

Revisar la configuración de ejecución

Para verificar tu aporte con tools/verify.py no necesitas configurar estas variables ni conectar Copilot o MCP. Revisa para qué sirven; el líder completa su .env antes de ejecutar python main.py desde main, después de integrar todos los PR, incluido Datos y persistencia si son siete. Conserva el .env copiado en el paso anterior.

Todos revisan; el líder configura su .env

Cómo realizar este paso

.env de S1 · para la ejecución del líder

Edita crewai/.env, junto a main.py. Sustituye los valores de ejemplo por los acordados con el instructor. MCP Projects es el servicio que consulta el backlog en GitHub; la URL y su clave deben corresponder al mismo servidor. GITHUB_OWNER y GITHUB_REPO ayudan al cliente a resolver los issues del backlog. El Project y el requerimiento se eligen al ejecutar main.py, no con estas variables.

Archivo

```
# Dirección del servidor MCP que permite leer GitHub Projects.
# localhost solo sirve si ese servidor corre en esta computadora.
PROJECTS_MCP_URL=http://localhost:8081
```

```
# Clave de acceso al servidor MCP, proporcionada por el instructor.
# No es un token de GitHub ni una clave de Copilot.
PROJECTS_MCP_API_KEY=<clave-del-MCP>

# Usuario u organización y repositorio donde está el issue.
# Ejemplo: https://github.com/mi-equipo/workshop-requirements-api/issues/12
GITHUB_OWNER=mi-equipo
GITHUB_REPO=workshop-requirements-api
```

Qué entra y qué sale de S1

El objetivo de S1 es coordinar los seis agentes para generar una propuesta a partir de un requerimiento, con revisión humana por paso. main.py no lee el clon de reqflow-workspace ni aplica allí los archivos generados. El equipo debe revisarlos y adaptar lo que corresponda antes de incorporarlo al producto. El repositorio del issue y el producto afectado pueden ser distintos; confírmenlo en el requerimiento.

Nota

```
Entrada: un requerimiento del GitHub Project acordado.
Ejecuta el líder: python main.py desde crewai/, después de integrar todos los PR, incluido Datos y persistencia si son siete.
Durante la ejecución: elige el Project y el requerimiento, y revisa cada resultado antes de aprobarlo.
Salida: especificaciones y código generado en crewai/output/.
Producto de los días anteriores: permanece en reqflow-workspace/<repo-del-producto>/.
```

Completado

S1 · Paso 6 de 22

Crear tu rama y editar tu agente

Cada persona edita solo agents/backend.py: rol, objetivo, contexto y la tarea de su agente. Más abajo encontrarás las instrucciones mínimas de cada rol. Copia las de tu

rol, adáptalas al Issue y úsalas en INSTRUCTIONS. Los entregables descritos los produce la ejecución integrada posterior; tu PR modifica solo el archivo Python del agente.

Cada integrante

> Cómo realizar este paso

Rama de tu rol

Con el árbol de trabajo limpio, actualiza tu main desde main del líder y crea tu rama. Sirve para el líder y para integrantes con forks; no depende de que main del fork ya esté sincronizado.

Comando

```
git status --short
git switch main &&
git fetch https://github.com/user-bcp-student47/crewai.git main &&
git merge --ff-only FETCH_HEAD &&
git switch -c agents/backend
```

Abrir tu archivo

Sustituye backend por tu rol en el equipo: architect, backend, frontend, testing, security o chapter; solo el equipo de siete puede elegir data. Por ejemplo, Backend usa code agents/backend.py. Abre el archivo en VS Code o en el editor que prefieras.

Comando

```
code agents/backend.py
```

Architect: instrucciones mínimas

Edita agents/architect.py. Usa estas instrucciones como base para INSTRUCTIONS y adapta role, goal, backstory y expected_output al Issue del equipo. Conserva KEY, SPEC_FILE y la firma de build_task.

Prompt

```
Analiza el Issue, sus criterios de aceptación y las interfaces del producto.
Define componentes, responsabilidades y límites del cambio.
Especifica entradas, respuestas y errores de la API, con ejemplos.
Relaciona cada criterio con el componente que lo resuelve. Declara supuestos y
preguntas pendientes.
Entrega user_story.md con componentes, contrato de API y criterios verificables.
```

Datos y persistencia: instrucciones mínimas

Edita agents/data.py. Solo para el equipo de siete en CrewAI. Usa estas instrucciones como base para INSTRUCTIONS y adapta role, goal, backstory y expected_output al Issue del equipo. Conserva KEY, SPEC_FILE y la firma de build_task.

Prompt

```
Parte de la arquitectura aprobada y del esquema existente.
Enumera entidades, atributos, tipos y cardinalidades.
Define claves primarias y foráneas, nulabilidad, unicidad, índices y reglas de
integridad.
Propón migraciones compatibles con los datos existentes, su orden y las
comprobaciones previas y posteriores.
Incluye pruebas de relaciones, duplicados y valores inválidos, y un plan de
reversión o recuperación.
Coordina el contrato de persistencia con Backend. Propón el SQL sin ejecutar
migraciones ni borrar datos.
Entrega data_spec.md y propuestas SQL en bloques de código cuya primera línea sea -
- nombre.sql. Incluye verificaciones de integridad y recuperación.
```

Backend: instrucciones mínimas

Edita agents/backend.py. Usa estas instrucciones como base para INSTRUCTIONS y adapta role, goal, backstory y expected_output al Issue del equipo. Conserva KEY, SPEC_FILE y la firma de build_task.

Prompt

```
Implementa los endpoints y servicios del plan, respetando el contrato de API.
Valida las entradas y devuelve errores coherentes.
Define acceso a datos y transacciones. En equipos de seis cubre también esquema y
migraciones; con Datos, consume su contrato sin duplicar sus migraciones.
Incluye un caso válido, un error y comandos para comprobarlos.
Entrega backend_spec.md y código Java con el formato de extracción que ya exige
build_task.
```

Frontend: instrucciones mínimas

Edita agents/frontend.py. Usa estas instrucciones como base para INSTRUCTIONS y adapta role, goal, backstory y expected_output al Issue del equipo. Conserva KEY, SPEC_FILE y la firma de build_task.

Prompt

```
Implementa el recorrido del usuario según el contrato de API aprobado.
Define campos, validaciones, acciones y mensajes.
Cubre estados de carga, vacío, éxito y error, y evita envíos duplicados.
Conserva etiquetas accesibles, navegación por teclado y foco visible.
Incluye cómo comprobar el recorrido y sus errores.
Entrega frontend_spec.md y código Angular con el formato de extracción que ya exige
build_task.
```

Testing: instrucciones mínimas

Edita agents/testing.py. Usa estas instrucciones como base para INSTRUCTIONS y adapta role, goal, backstory y expected_output al Issue del equipo. Conserva KEY,

SPEC_FILE y la firma de build_task.

Prompt

```
Relaciona cada criterio de aceptación con una prueba y su resultado esperado.
Incluye casos positivos, negativos, de borde e integración.
Detalla preparación, datos de prueba, comandos y limpieza.
Comprueba los contratos de API y persistencia y las correcciones propuestas.
Marca NO EJECUTADO cuando no tengas una salida real. No inventes resultados.
Entrega testing_report.md, pruebas y comandos reproducibles.
```

Security: instrucciones mínimas

Edita agents/security.py. Usa estas instrucciones como base para INSTRUCTIONS y adapta role, goal, backstory y expected_output al Issue del equipo. Conserva KEY, SPEC_FILE y la firma de build_task.

Prompt

```
Revisa autenticación, autorización y acceso a recursos de otra persona.
Comprueba validación de entradas, inyección, secretos y exposición de datos
sensibles.
Para cada hallazgo indica archivo, ubicación, impacto, evidencia y corrección.
Propón una comprobación reproducible de la corrección.
Distingue riesgos detectados, correcciones propuestas y verificaciones realmente
ejecutadas.
Entrega security_report.md con hallazgos, correcciones y comprobaciones.
```

Chapter: instrucciones mínimas

Edita agents/chapter.py. Usa estas instrucciones como base para INSTRUCTIONS y adapta role, goal, backstory y expected_output al Issue del equipo. Conserva KEY, SPEC_FILE y la firma de build_task.

Prompt

```
Contrasta cada criterio de aceptación con arquitectura, persistencia,
implementación, pruebas y seguridad.
Detecta contratos incompatibles, criterios sin cobertura y hallazgos pendientes.
Para cada observación indica responsable, cambio requerido y cómo verificarlo.
Justifica el veredicto con evidencia y señala las pruebas pendientes.
Conserva las aprobaciones humanas; tu informe no las sustituye.
Entrega chapter_review.md con VEREDICTO: APROBADO o RECHAZADO, motivos y
correcciones.
```

Completado

S1 · Paso 7 de 22

Verificar sin servicios externos

verify.py comprueba los flujos de seis y siete roles con respuestas simuladas. No llama a Copilot ni a MCP. Cada persona verifica su rama antes del commit y del PR; main.py se ejecuta después de que el líder integre todas las entregas, incluida Datos y persistencia si son siete.

Cada integrante

Cómo realizar este paso

Checks offline

Comprueba los flujos de seis y siete roles, sus artefactos, contexto y reanudación sin credenciales.

Comando

```
python tools/verify.py
```

Completado

S1 · Paso 8 de 22

Commit y push de tu rama

Después de que verify.py pase en tu rama, publica solo tu archivo agents/backend.py. Los integrantes hacen push a su fork; el líder hace push a su repo original. Conserva la salida de la prueba y el SHA del commit.

Cada integrante

> Cómo realizar este paso

Revisar el cambio

Confirma que solo cambió tu archivo antes de crear el commit.

Comando

```
git status
git diff agents/backend.py
```

Commit y push

Crea el commit con tu archivo y publica la rama en origin. Para integrantes, origin es su fork; para el líder, su repo original.

Comando

```
git add agents/backend.py
git commit -m "Ajusta el agente backend"
git push -u origin agents/backend
git rev-parse HEAD
```

Completado

S1 · Paso 9 de 22

Abrir el PR y pedir revisión

Cada persona, incluido el líder, abre su PR hacia main del repo del líder con la evidencia offline. Otra persona lo revisa. El líder integra todos los PR aprobados antes de ejecutar main.py.

Cada integrante · revisa un compañero

Cómo realizar este paso

Compañero: revisar el PR

Con tu trabajo guardado, revisa el diff del PR y ejecuta verify.py en esa rama. Aprueba solo si entiendes el cambio y la prueba pasa; usa --request-changes -b "motivo" si necesita ajustes. Al terminar vuelve a tu rama. No ejecutes main.py durante esta revisión.

Comando

```
git status --short
gh pr diff <n> --repo user-bcp-student47/crewai
gh pr checkout <n> --repo user-bcp-student47/crewai &&
python tools/verify.py &&
gh pr review <n> --repo user-bcp-student47/crewai --approve
git switch agents/backend
```

Completado

S1 · Paso 10 de 22

Líder: ejecutar el Crew desde main

Solo el líder opera esta ejecución; todo el equipo participan en la revisión. Con todos los PR revisados e integrados, el líder actualiza main en su clon crewai y vuelve a

ejecutar verify.py. Solo si pasa inicia main.py con su .env y sesión de Copilot. El equipo revisa cada salida e interviene en las aprobaciones humanas.

El líder ejecuta; todo el equipo revisa

Cómo realizar este paso

Este paso lo hace tu líder @user-bcp-student47. Revisa las instrucciones de este paso y espera a que lo complete para continuar con tu equipo.

Completado

S1 · Paso 11 de 22

Revisar aprobaciones humanas y evidencia

Durante la ejecución integrada, cada responsable revisa la salida de su rol y el equipo decide si continuar, reintentar o detener. El líder introduce la respuesta en la terminal. Después, todas las personas conservan evidencia de su entrega y del resultado integrado.

El líder opera; cada persona conserva evidencia

Cómo realizar este paso

Opciones de la aprobación humana

Estas decisiones se toman durante main.py desde main integrado. El líder introduce la opción acordada tras la revisión del responsable; s continúa, r reejecuta el paso y n detiene el flujo. Una aprobación humana no sustituye la salida real de pruebas.

Nota

```
s  aprobar y continuar
r  reintentar con feedback
```

```
n  rechazar y detener el flujo
```

Artefactos de la ejecución

Lista los informes: user_story, backend_spec, frontend_spec, testing_report, security_report y chapter_review.

Comando

```
ls output/specs
```

Completado

S2 · Paso 12 de 22

Revisa tu equipo y prepara el entorno

Revisa tu equipo, tu líder asignado y tu participación. Comprueba tu entorno en Bash. Cada equipo tiene seis perfiles, o siete con Datos y persistencia de LangGraph: un archivo por persona.

Todo el equipo

Cómo realizar este paso

Comprobar herramientas en Bash

Confirma que tienes Python 3.12, Git y GitHub CLI.

Comando

```
python --version
git --version
gh --version
```

Sesión de GitHub CLI

GitHub CLI crea, clona y abre los PR del equipo. Debe estar autenticado con tu cuenta personal.

Comando

```
gh auth status
```

Completado

S2 · Paso 13 de 22

Líder: crear guardrails desde la plantilla

El líder crea una sola vez el repo guardrails en su cuenta personal a partir de NTTData-Academy/guardrails. Si ya existe, verifica que sea el repo del equipo y reutilízalo. Los integrantes esperan al líder y crean un fork de su repo.

Solo el líder

Cómo realizar este paso

Este paso lo hace tu líder @user-bcp-student47. Revisa las instrucciones de este paso y espera a que lo complete para continuar con tu equipo.

Completado

S2 · Paso 14 de 22

Elegir tu rol y crear tu fork

Cada persona guarda su rol en «Antes de empezar». El líder ya creó guardrails desde la plantilla en el paso anterior y usa ese repositorio. Cada integrante crea un fork del repositorio del líder en su propia cuenta de GitHub.

Líder e integrantes

> Cómo realizar este paso

Cómo trabajamos en equipo

El grupo de siete añade Datos y persistencia en agents/data.py. El líder configura GUARDRAILS_INCLUDE_DATA=true en .env antes de un flujo nuevo. Datos corre después de Arquitectura y antes de Backend, con aprobación humana; entra también al ciclo de correcciones de Chapter. Cada persona entrega su propio archivo con su PR. Los equipos de seis mantienen seis nodos.

Nota

```
architect → agents/architect.py: Define componentes, límites y contrato de API:
entradas, respuestas y errores, alineados con los criterios de aceptación.
data → agents/data.py: Define tablas, relaciones, restricciones y migraciones:
claves primarias y foráneas, nulabilidad, unicidad, índices e integridad; conserva
los datos existentes y propone reversión o recuperación. Solo para el equipo de
siete en CrewAI y Guardrails.
backend → agents/backend.py: Implementa servicios, endpoints, validaciones y acceso
a datos según los contratos aprobados de Arquitectura y Datos. En equipos de seis
también cubre la persistencia.
frontend → agents/frontend.py: Implementa la interacción y el consumo de API, con
estados de carga, vacío, error y accesibilidad.
testing → agents/testing.py: Convierte criterios de aceptación en pruebas
positivas, negativas, de borde e integración; propone comandos y distingue
resultados reales de NO EJECUTADO.
security → agents/security.py: Revisa autenticación, autorización, entradas,
secretos, datos sensibles e inyección; entrega hallazgos, correcciones y
comprobaciones.
chapter → agents/chapter.py: Contrasta requerimiento, arquitectura, datos, código,
pruebas y seguridad; emite un veredicto con motivos y correcciones. Conserva las
aprobaciones humanas.
```

Completado

S2 · Paso 15 de 22

Clonar guardrails, preparar Python y ubicar el producto

El líder usa su repo guardrails y cada integrante su fork del líder. Abre una nueva terminal desde el acceso Workshop - Bash creado el Día 1. Empiezas en tu carpeta del curso, <matricula>-<nombre>/. Ahí clonarás guardrails, junto a reqflow-workspace/.

Cada persona

Cómo realizar este paso

Dónde queda cada repositorio

Esta es la estructura que tendrás al clonar los dos repositorios del Día 4. Ahora vas a trabajar con guardrails/. Cada clon tiene su propio .venv y .env. Conserva los productos dentro de reqflow-workspace/.

Nota

```
<matricula>-<nombre>/
├── reqflow-workspace/
│   └── repositorios del producto de los días anteriores
├── crewai/
└── guardrails/
```

Clonar el repo de mi cuenta

Ejecuta este bloque desde tu carpeta <matricula>-<nombre>/. Descarga guardrails desde tu cuenta de GitHub, entra en el clon y muestra su ruta. Si la descarga falla, el bloque se detiene.

Comando

```
gh repo clone user-bcp-student09/guardrails &&
cd guardrails &&
pwd
```

Conectar mi fork con el repo del líder

origin es tu fork para hacer push. upstream es el repo del líder, destino de los PR. Si upstream ya existe, comprueba su URL con git remote -v en lugar de añadirlo otra vez.

Comando

```
git remote get-url upstream >/dev/null 2>&1 || git remote add upstream
https://github.com/user-bcp-student47/guardrails.git
git remote -v
```

Crear el entorno en Bash

Crea .venv con Python 3.12 solo si falta, activa el entorno existente, instala las dependencias y copia .env.example solo si aún no tienes .env.

Comando

```
[ -d .venv ] || python -m venv .venv
if [ -f .venv/Scripts/activate ]; then
  source .venv/Scripts/activate
else
  source .venv/bin/activate
fi
python -m pip install -r requirements.txt
[ -f .env ] || cp .env.example .env
```

Ubicar el producto de los días anteriores

Desde la raíz del clon de esta sesión, comprueba el producto existente. workshop-auth-backend es el ejemplo: sustituye el nombre por el repo que corresponde al issue. Conserva su historial y los cambios de los días previos; este paso no crea ni clona otro producto.

Comando

```
git -C ../reqflow-workspace/workshop-auth-backend remote -v
git -C ../reqflow-workspace/workshop-auth-backend status --short --branch
```

Completado

S2 · Paso 16 de 22

Revisar la configuración de ejecución

Para verificar tu aporte con tools/verify.py no necesitas configurar estas variables ni conectar Copilot o MCP. Revisa para qué sirven; el líder completa su .env antes de ejecutar python main.py desde main, después de integrar todos los PR, incluido Datos y persistencia si son siete. Conserva el .env copiado en el paso anterior.

Todos revisan; el líder configura su .env

Cómo realizar este paso

Autenticar GitHub CLI

Reutiliza la sesión iniciada al preparar el entorno. Si falta, ejecuta gh auth login. El líder la usa para publicar el PR del producto; todos la necesitan para sus entregas de agentes.

Comando

```
gh auth status
```

Variables de .env para S2 · producto existente

GITHUB_OWNER y GITHUB_REPO apuntan al producto usado en los días anteriores; workshop-auth-backend es el ejemplo. Cambia ambos si el issue afecta a otro repo. El líder debe poder publicar una rama y un PR en ese producto; GIT_HEAD_BRANCH es la rama del refactor, distinta de agents/backend.

Archivo

```
GITHUB_OWNER=<owner-del-producto>
GITHUB_REPO=workshop-auth-backend
GITHUB_ISSUE=<número-del-issue>
GIT_BASE_BRANCH=main
GIT_HEAD_BRANCH=feature/equipo-XX/workshop-refactor
```

Producto existente y copia de ejecución de S2

Conserva el clon de reqflow-workspace. Antes de ejecutar, el líder comprueba que los cambios previos necesarios estén integrados y publicados en la rama base del producto. La plantilla de S2 trabaja en .tmp y no permite configurar directamente el clon existente: esa copia automática es parte del runtime. No crees otro repo de producto ni lo clones manualmente para este laboratorio.

Nota

```
Producto del equipo: el mismo repo remoto usado en los días anteriores.
Clon existente: ../reqflow-workspace/<repo-del-producto>/.
Copia interna del runtime: guardrails/.tmp/<GITHUB_REPO>/, creada o actualizada
automáticamente por main.py.
Ejemplo: workshop-auth-backend; ajusta GITHUB_OWNER y GITHUB_REPO al issue elegido.
```

Completado

S2 · Paso 17 de 22

Crear tu rama y editar tu nodo

Cada persona edita solo agents/backend.py: el prompt y los entregables de su nodo. Más abajo encontrarás las instrucciones mínimas de cada rol. Copia las de tu rol, adáptalas al Issue y úsalas en INSTRUCTIONS. Los entregables descritos los produce la ejecución integrada posterior; tu PR modifica solo el archivo Python del agente.

Cada integrante

Cómo realizar este paso

Rama de tu rol

Con el árbol de trabajo limpio, actualiza tu main desde main del líder y crea tu rama. Sirve para el líder y para integrantes con forks; no depende de que main del fork ya esté sincronizado.

Comando

```
git status --short
git switch main &&
git fetch https://github.com/user-bcp-student47/guardrails.git main &&
git merge --ff-only FETCH_HEAD &&
git switch -c agents/backend
```

Abrir tu archivo

Sustituye backend por tu rol en el equipo: architect, backend, frontend, testing, security o chapter. Por ejemplo, Backend usa code agents/backend.py. Abre el archivo en VS Code o en el editor que prefieras.

Comando

```
code agents/backend.py
```

Architect: instrucciones mínimas

Edita agents/architect.py. Usa estas instrucciones como base para INSTRUCTIONS y adáptalas al Issue del equipo. Conserva la firma del nodo, las claves del estado y el formato de salida.

Prompt

```
Analiza el Issue, sus criterios de aceptación y las interfaces del producto.
Define componentes, responsabilidades y límites del cambio.
Especifica entradas, respuestas y errores de la API, con ejemplos.
Relaciona cada criterio con el componente que lo resuelve. Declara supuestos y
preguntas pendientes.
Devuelve ## Analysis y ## Plan en líneas independientes, como exige architect_node.
Incluye diagnóstico, archivos afectados y orden de los cambios.
```

Datos y persistencia: instrucciones mínimas

Edita agents/data.py. Solo para el equipo de siete en Guardrails. Usa estas instrucciones como base para INSTRUCTIONS y adáptalas al Issue del equipo. Conserva la firma del nodo, la clave out_data y el formato de salida.

Prompt

```
Parte de la arquitectura aprobada y del esquema existente.
Enumera entidades, atributos, tipos y cardinalidades.
Define claves primarias y foráneas, nulabilidad, unicidad, índices y reglas de
integridad.
Propón migraciones compatibles con los datos existentes, su orden y las
comprobaciones previas y posteriores.
Incluye pruebas de relaciones, duplicados y valores inválidos, y un plan de
reversión o recuperación.
Coordina el contrato de persistencia con Backend. Propón el SQL sin ejecutar
migraciones ni borrar datos.
Devuelve el contrato de datos y propuestas SQL con ### ruta/archivo.sql seguido de
su bloque de código. Incluye verificaciones de integridad y recuperación e
```

```
incorpora las observaciones del Chapter en las siguientes rondas. La CLI guarda el
informe en data_report.md.
```

Backend: instrucciones mínimas

Edita agents/backend.py. Usa estas instrucciones como base para INSTRUCTIONS y adáptalas al Issue del equipo. Conserva la firma del nodo, las claves del estado y el formato de salida.

Prompt

```
Implementa los endpoints y servicios del plan, respetando el contrato de API.
Valida las entradas y devuelve errores coherentes.
Define acceso a datos y transacciones. En equipos de seis cubre también esquema y
migraciones; con Datos, consume su contrato sin duplicar sus migraciones y conserva
los datos existentes.
Incluye un caso válido, un error y comandos para comprobarlos.
Devuelve los cambios con ### ruta/archivo.ext seguido de su bloque de código.
Incorpora las observaciones del Chapter en las siguientes rondas.
```

Frontend: instrucciones mínimas

Edita agents/frontend.py. Usa estas instrucciones como base para INSTRUCTIONS y adáptalas al Issue del equipo. Conserva la firma del nodo, las claves del estado y el formato de salida.

Prompt

```
Implementa el recorrido del usuario según el contrato de API aprobado.
Define campos, validaciones, acciones y mensajes.
Cubre estados de carga, vacío, éxito y error, y evita envíos duplicados.
Conserva etiquetas accesibles, navegación por teclado y foco visible.
Incluye cómo comprobar el recorrido y sus errores.
```

```
Devuelve los cambios con ### ruta/archivo.ext seguido de su bloque de código.
Incorpora las observaciones del Chapter en las siguientes rondas.
```

Testing: instrucciones mínimas

Edita agents/testing.py. Usa estas instrucciones como base para INSTRUCTIONS y adáptalas al Issue del equipo. Conserva la firma del nodo, las claves del estado y el formato de salida.

Prompt

```
Relaciona cada criterio de aceptación con una prueba y su resultado esperado.
Incluye casos positivos, negativos, de borde e integración.
Detalla preparación, datos de prueba, comandos y limpieza.
Comprueba los contratos de API y persistencia y las correcciones propuestas.
Marca NO EJECUTADO cuando no tengas una salida real. No inventes resultados.
Devuelve un informe con cobertura por criterio, pruebas propuestas, comandos y
resultados reales o NO EJECUTADO.
```

Security: instrucciones mínimas

Edita agents/security.py. Usa estas instrucciones como base para INSTRUCTIONS y adáptalas al Issue del equipo. Conserva la firma del nodo, las claves del estado y el formato de salida.

Prompt

```
Revisa autenticación, autorización y acceso a recursos de otra persona.
Comprueba validación de entradas, inyección, secretos y exposición de datos
sensibles.
Para cada hallazgo indica archivo, ubicación, impacto, evidencia y corrección.
Propón una comprobación reproducible de la corrección.
Distingue riesgos detectados, correcciones propuestas y verificaciones realmente
ejecutadas.
```

```
Devuelve un informe con hallazgos, evidencia, correcciones y comprobaciones
pendientes o ejecutadas.
```

Chapter: instrucciones mínimas

Edita agents/chapter.py. Usa estas instrucciones como base para INSTRUCTIONS y adáptalas al Issue del equipo. Conserva la firma del nodo, las claves del estado y el formato de salida.

Prompt

```
Contrasta cada criterio de aceptación con arquitectura, persistencia,
implementación, pruebas y seguridad.
Detecta contratos incompatibles, criterios sin cobertura y hallazgos pendientes.
Para cada observación indica responsable, cambio requerido y cómo verificarlo.
Justifica el veredicto con evidencia y señala las pruebas pendientes.
Conserva las aprobaciones humanas; tu informe no las sustituye.
Emite VEREDICTO: APROBADO, APROBADO CON OBSERVACIONES o RECHAZADO. Verifica las
observaciones de rondas anteriores y detalla las que deben volver a desarrollo.
```

Completado

S2 · Paso 18 de 22

Verificar sin servicios externos

verify.py recorre el grafo real con respuestas simuladas del modelo. No llama a Copilot, MCP ni GitHub. Cada persona verifica su rama antes del commit y del PR; main.py se ejecuta después de que el líder integre todas las entregas, incluida Datos y persistencia si son siete.

Cada integrante

Cómo realizar este paso

Checks offline

Comprueba pausas humanas, los flujos de seis y siete roles, feedback, límite de rondas, reanudación y guardrails sin credenciales.

Comando

```
python tools/verify.py
```

Completado

S2 · Paso 19 de 22

Commit y push de tu rama

Después de que verify.py pase en tu rama, publica solo tu archivo agents/backend.py. Los integrantes hacen push a su fork; el líder hace push a su repo original. Conserva la salida de la prueba y el SHA del commit.

Cada integrante

Cómo realizar este paso

Revisar el cambio

Confirma que solo cambió tu archivo antes de crear el commit.

Comando

```
git status
git diff agents/backend.py
```

Commit y push

Crea el commit con tu archivo y publica la rama en origin. Para integrantes, origin es su fork; para el líder, su repo original.

Comando

```
git add agents/backend.py
git commit -m "Ajusta el nodo backend"
git push -u origin agents/backend
git rev-parse HEAD
```

Completado

S2 · Paso 20 de 22

Abrir el PR y pedir revisión

Cada persona, incluido el líder, abre su PR hacia main del repo del líder con la evidencia offline. Otra persona lo revisa. El líder integra todos los PR aprobados antes de ejecutar main.py.

Cada integrante · revisa un compañero

Cómo realizar este paso

Compañero: revisar el PR

Con tu trabajo guardado, revisa el diff del PR y ejecuta verify.py en esa rama. Aprueba solo si entiendes el cambio y la prueba pasa; usa --request-changes -b "motivo" si necesita ajustes. Al terminar vuelve a tu rama. No ejecutes main.py durante esta revisión.

Comando

```
git status --short
gh pr diff <n> --repo user-bcp-student47/guardrails
gh pr checkout <n> --repo user-bcp-student47/guardrails &&
python tools/verify.py &&
```

```
gh pr review <n> --repo user-bcp-student47/guardrails --approve
git switch agents/backend
```

Completado

S2 · Paso 21 de 22

Líder: ejecutar el flujo desde main

Solo el líder opera esta ejecución; todo el equipo participan en la revisión. Con todos los PR revisados e integrados, el líder actualiza main en su clon guardrails y vuelve a ejecutar verify.py. Solo si pasa inicia main.py con su .env y sesión de Copilot. El equipo revisa cada salida e interviene en las aprobaciones humanas.

El líder ejecuta; todo el equipo revisan

Cómo realizar este paso

Este paso lo hace tu líder @user-bcp-student47. Revisa las instrucciones de este paso y espera a que lo complete para continuar con tu equipo.

Completado

S2 · Paso 22 de 22

Revisar aprobaciones, evaluaciones y evidencia

Durante la ejecución integrada, cada responsable revisa la salida de su rol y el equipo decide si continuar, reintentar o detener. El líder introduce la respuesta en la terminal. Después, todas las personas conservan evidencia de su entrega y del resultado integrado.

El líder opera; cada persona conserva evidencia

Cómo realizar este paso

Opciones de la aprobación humana

Estas decisiones se toman durante main.py desde main integrado. El líder introduce la opción acordada tras la revisión del responsable; s continúa, r reejecuta el paso y n detiene el flujo. Una aprobación humana no sustituye la salida real de pruebas. En S2, r reejecuta el nodo; la plantilla no pide feedback libre en esa pausa. El ciclo de observaciones de Chapter se gestiona en el grafo.

Nota

```
s  aprobar y continuar
r  reejecutar el nodo
n  rechazar y detener el flujo
```

Evidencia de la ejecución

architecture_report, backend_refactor, frontend_refactor, testing_report, security_report y chapter_report; las rondas extra usan _r<N>. evidence/ guarda compliance, evals y la URL del PR.

Comando

```
ls output evidence
```